

Algorithmes de tri:

Tri par fusion

Continuité

```
# -----  
# L'ordre d'exécution dans les fonctions récursives,  
# à comprendre avant de d'analyser l'algorithme de tri par fusion :
```

```
def fct(n):  
    if n<=1:  
        return 1  
    else:  
        fct(n-1)  
        print('bonjour :', n)
```

```
fct(4)
```

```
bonjour : 2  
bonjour : 3  
bonjour : 4
```

L'ordre de l'exécution
de ce programme

Exécution :

```
fct(4)  # 1er appel  
|  
v  
fct(3) 3 c'est n-1  ## et print('bonjour :', 4), 4 c'est n, sera mise en attente de revenir de l'appel fct(3)  
|  
v  
fct(2) 2 c'est n-1  ## et print('bonjour :', 3), 3 c'est n? sera mise en attente de revenir de l'appel fct(2)  
|  
v  
fct(1) 1 c'est n-1  ## et print('bonjour :', 2), 2 c'est n, sera mise en attente de revenir de l'appel fct(1)  
|  
v  
## ceci retourne 1,
```

et donc fct(1) vient de terminer, on print donc bonjour 2

puis, on remonte vers fct(2) qui vient de terminer aussi,
et on print bonjour 3

puis, on remonte vers fct(3) qui vient de terminer aussi,
et on print bonjour 4

Toujours le cas de base
qui s'exécute le premier,
puis on remonte pas à pas
jusqu'au 1^{er} appel de la
fonction.

Diviser et trier en utilisant la récursivité.

Cet algorithme est plus rapide que les trois autres algorithmes de tri précédents.

```
def tri_fusion(arr):
    if len(arr) > 1:
        milieu = len(arr) // 2

        moitie_gauche = arr[:milieu + len(arr) % 2] # Prend la première moitié de la liste
                                                    # inclut le milieu si la longueur de la liste est impaire

        moitie_droite = arr[milieu + len(arr) % 2:] # Prend la deuxième moitié de la liste

        tri_fusion(moitie_gauche) # Appel récursif pour trier la moitié gauche
        tri_fusion(moitie_droite) # Appel récursif pour trier la moitié droite

        i = j = k = 0
        ## 1ère boucle while : affecte à arr[0] = Le petit élément
                                #   arr[1] = Le petit élément dans ce qui reste
                                #   arr[2] = Le petit élément dans ce qui reste,
                                #   etc.
        while i < len(moitie_gauche) and j < len(moitie_droite):
            if moitie_gauche[i] < moitie_droite[j]:
                arr[k] = moitie_gauche[i]
                i += 1
            else:
                arr[k] = moitie_droite[j]
                j += 1
            k += 1

        # Les deuxième et troisième boucles while sont utilisées pour gérer les cas
        # où une des moitiés (gauche ou droite) aurait encore des éléments restants
        # après la fusion de la première boucle. Elles sont nécessaires car les moitiés
        # peuvent ne pas avoir la même longueur. Ces boucles ajoutent simplement les
        # éléments restants à la fin de la liste arr

        while i < len(moitie_gauche):
            arr[k] = moitie_gauche[i]
            i += 1
            k += 1

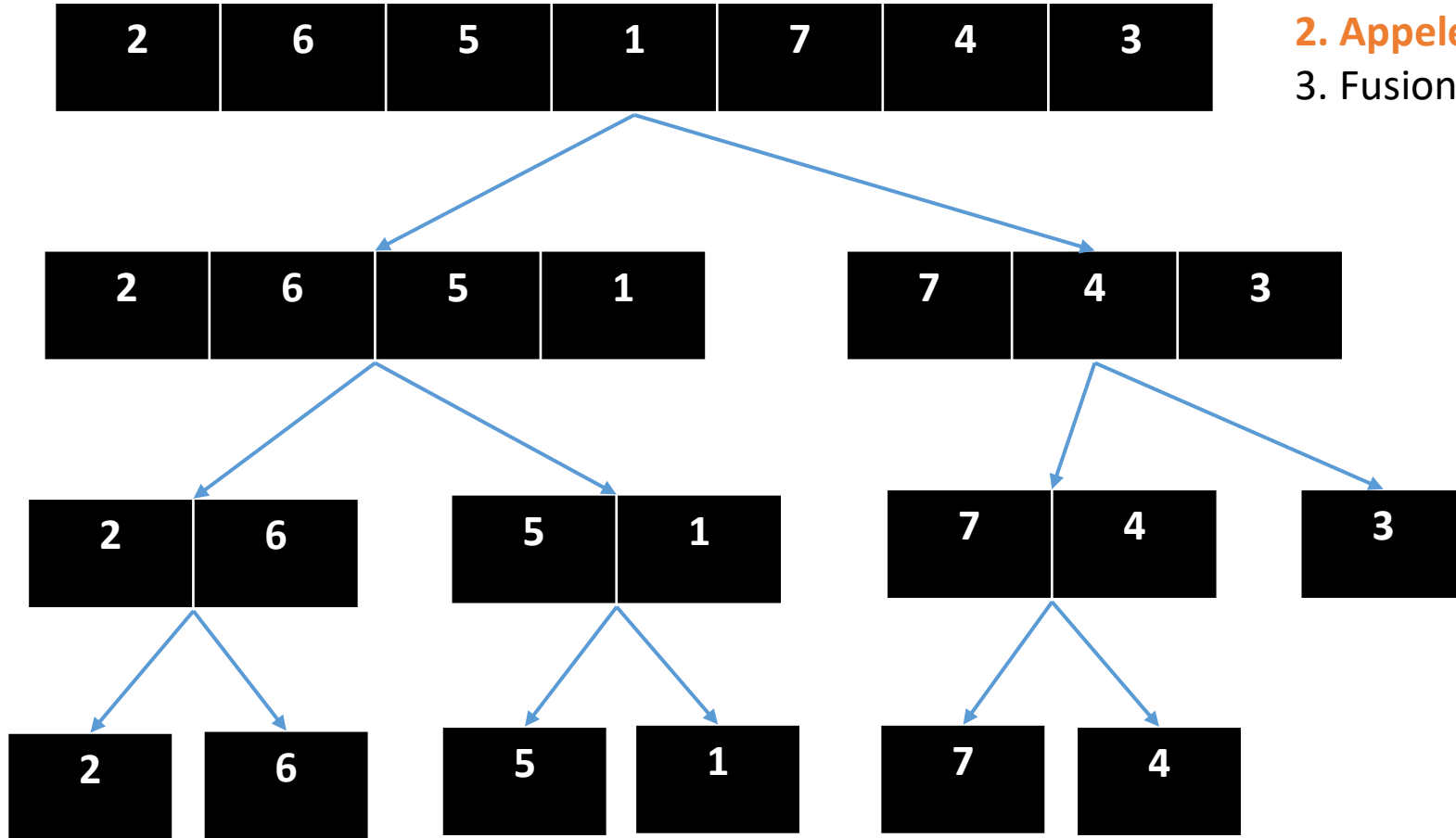
        while j < len(moitie_droite):
            arr[k] = moitie_droite[j]
            j += 1
            k += 1

    return arr
```

Comment ça marche, pas à pas

Cet algorithme se compose de 3 étapes principales:

1. Diviser la liste en deux parties
2. Appeler la fonction `tri_fusion` elle-même (récursivité)
3. Fusionner les deux parties en une seule liste (triée)

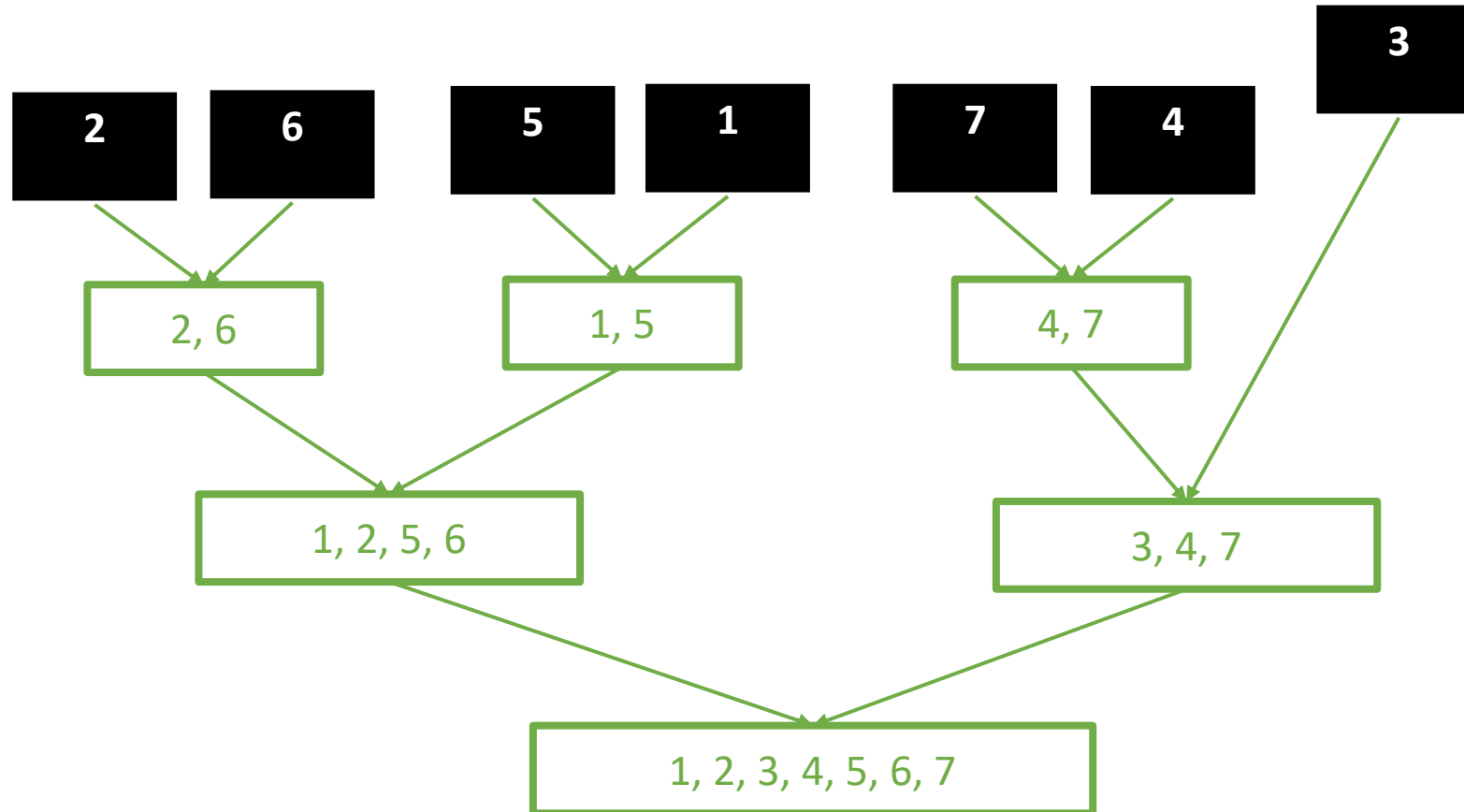


Si la taille est impaire, on choisit une partie qui sera plus grande que l'autre, Mais, si, par exemple, nous avons choisi la partie gauche qui sera plus grande, on doit rester consistant durant tout l'algorithme (c.à.d., toujours la partie gauche doit être la plus grande pour les autres divisions dans le cas où la taille est impaire), comme pour la sous liste [7, 4, 3] On continue la division, jusqu'à ce que toutes les sous-listes n'ont qu'un seul élément.

maintenant, on
doit les
refusionner,
mais de
manière triée.

Comment ça marche, pas à pas

Donc voici les éléments individuels que nous avons eu après division



mg : moitié gauche
md : moitié droite

[2] [6] 1ère while, itération 1 : 2<6 ? oui arr[k] = 2 i += 1 k += 1 2ème while, itération 1 : arr[k] = 6 résultat : arr = [1, 6]	[5] [1] même logique résultat : arr = [1, 6]	[7] [4] même logique résultat : arr = [4, 7]	[3] ici, on rentre pas à la 1ère while, car, i < len(mg) est True mais, j n'est pas < len(md) car len(md) = 0 On rentre directement à la 2ème while, 1ère itération : while i < len(mg) : arr[k] = mg[i] i += 1 FIN, car i n'est plus < len(mg) résultat : arr = [3]
---	---	---	---

Continuité :

[2, 6] [1, 5]

1ère while, _____

itération 1 : *****

2 < 1 ? Non

arr[k] = 1

j += 1

itération 2 : *****

2 < 5 ? oui

arr[k] = 2

i += 1

itération 3 : *****

6 < 5 ? Non

arr[k] = 5

j += 1

FIN 1ère while, car j n'est plus < len(md)

2ème while, _____

itération 1 : *****

arr[k] = 6

résultat :

arr = [1, 2, 5, 6]

[4, 7] [3]

1ère while, _____

itération 1 : *****

4 < 3 ? Non

arr[k] = 3

j += 1

FIN 1ère while, car j n'est plus < len(md)

2ème while, _____

itération 1 : *****

arr[k] = 4

itération 2 : *****

arr[k] = 7

résultat :

arr = [3, 4, 7]

Algorithmes de recherche:

Recherche linéaire

L'algorithme de recherche le plus simple et intuitif

```
a = [1, 3, 2, 8, 5, 6, 9, 10, 12, 1, 12, 21]
```

```
def recherche_lineaire(arr, cible):  
    for i in range(len(arr)):  
        if arr[i] == cible:  
            return i # Indice de l'élément cible  
    return -1 # Cible non trouvée
```

```
recherche_lineaire(a, 12)
```

```
8
```

```
a.index(10) #cette méthode prédéfinie aussi utilise la recherche linéaire
```

Algorithmes de recherche:

Recherche binaire (ou dichotomique)

```
def recherche_binaire(arr, cible):  
    bas, haut = 0, len(arr) - 1  
    while bas <= haut:  
        milieu = (bas + haut) // 2  
        if cible == arr[milieu]:  
            return milieu # Indice de l'élément cible  
        elif cible > arr[milieu] :  
            bas = milieu + 1  
        else: ## cible < arr[milieu]  
            haut = milieu - 1  
    return -1 # Cible non trouvée
```

```
a = [3, 7, 9, 10, 12, 16, 20, 32]  
res = recherche_binaire(a, 20)  
res
```

Comment ça marche

```
## supposons que nous sommes entrain de chercher le nombre 20
```

```
a = [3, 7, 9, 10, 12, 16, 20, 32]
```

```
a = [3, 7, 9, 10, 12, 16, 20, 32]  ## bas = 0, haut = len(a)-1 = 7
                                   ## comparer 20 avec le nombre en milieu, (bas + haut) // 2 = 3
                                   ## 20 == a[3] ? non
                                   ## 20 > a[4] ? oui --> chercher dans la moitié droite
```

```
a = [3, 7, 9, 10, 12, 16, 20, 32]  ## bas = milieu + 1 = 3+1 = 4 (moitié droite = 4 à 7)
#a = [12, 16, 20, 32]              ## milieu = (bas + haut) // 2 = (4+7) // 2 = 11//2 = 5
                                   ## 20 == a[5] ? Non
                                   ## 20 > a[5] ? oui --> chercher dans la moitié droite
```

```
a = [3, 7, 9, 10, 12, 16, 20, 32]  ## bas = milieu + 1 = 5+1 = 6 (moitié droite = 6 à 7)
#a = [20, 32]                     ## milieu = (bas + haut) // 2 = (6+7) // 2 = 13//2 = 6
                                   ## 20 == a[6] ? Oui --> cible trouvé
```

```
## et donc nous l'avons trouvé dans 3 étapes seulement
```

```
## en comparaison avec la recherche linéaire qui compare tous les éléments de la liste
```

Notez : dans la recherche binaire, la liste doit être triée en préalable